

RATING PREDICTION

CMP2003 Data Structures and Algorithms (C++) Project
Due Date: Dec 29, 2024

1 The Problem

Recommender systems help people find items of interest by making personalized recommendations according to their preferences. For example, a recommender system can make personalized recommendations of items such as movies, books, hotels, or music to people. In order to model users' preferences their past interactions such as product views, ratings, and purchases are used. In the recommender systems area of research (which is a subfield of machine learning and information retrieval) different algorithms have been developed which are currently being used by many large companies.

In this project you will implement neighborhood based collaborative filtering (NBCF) algorithms in order to make predictions for movie ratings of people. There are two main types of NBCF algorithms: user-based (UBCF) and item-based (IBCF). Let us describe each with an example dataset.

2 UBCF and IBCF

One of the fundamental problems in recommender systems is to predict the rating of a user for a particular item. For example, given the dataset in Table 1, what might be the rating of User 2 for Movie 3? If we can predict this rating then, if it is a high value like 4 or 5, we can decide to recommend Movie 3 to User 2.

Table 1: An example dataset.

	Movie 1	Movie 2	Movie 3	Movie 4	Movie 5
User 1	1	2	1	1	5
User 2	2	3		4	5
User 3		5	4	4	2
User 4	4	4	3	2	3
User 5	3		5	2	4
User 6	4	2	3	5	4

In its simplest form, UBCF works as follows: in order to predict the rating of user u to item i , we first find the most similar k users to u (who rated i) and predict the rating as the average ratings of these most similar k users on item i . For finding the similarity between two users different methods can be used. In collaborative filtering we look at the ratings of other users and find users which have similar ratings. One popular method to find such a similarity is called cosine similarity which is given below:

$$\text{cosine}(A, B) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i \times B_i}{\sqrt{\sum_{i=1}^n A_i^2} \times \sqrt{\sum_{i=1}^n B_i^2}} \quad (1)$$

For example, to find the similarity between User 2 and User 5, we first find the movies that both users rated. The ratings of User 2 and User 5 to these movies are given by the vectors $A = [2, 4, 5]$ and

$B = [3, 2, 4]$. The cosine similarity between these vectors (hence the cosine similarity between User A and User B) is given by:

$$\text{cosine}(A, B) = \frac{2 \times 3 + 4 \times 2 + 5 \times 4}{\sqrt{2^2 + 4^2 + 5^2} \times \sqrt{3^2 + 2^2 + 4^2}} \quad (2)$$

IBCF, on the other hand, works as follows: in order to predict the rating of user u to item i , we first find the most similar k items to i (which are rated by user u) and predict the rating as the average rating of user u for these most similar k items. For similarity calculation, again cosine similarity can be used.

3 Evaluation

The dataset that will be given to you will be a text file and will have the following format (each line contains a rating of a particular user to a particular movie):

```
UserID, MovieID, Rating
10, 100, 5
10, 101, 4
10, 102, 5
11, 100, 5
11, 201, 4
...
```

For testing your performance you will be given a test set similar to the following:

```
UserID, MovieID
10, 200
10, 201
10, 202
11, 300
11, 301
...
```

As can be seen, the ratings are hidden, your task is to predict these ratings and make a submission to HackerRank web site. You have two objectives: Make the best predictions and do it in least amount of time. To make the best predictions you should predict the actual ratings as close as possible. The error metric for calculating your scores will be root mean squared error (RMSE), which is given below:

$$RMSE = \sqrt{\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}} \quad (3)$$

where y_i is the actual rating and $\hat{y}_i$ is the predicted rating. For example if $y = [2, 4, 5, 3]$ and $\hat{y} = [3, 4, 2, 2]$ then

$$RMSE = \sqrt{\frac{(2-3)^2 + (4-4)^2 + (5-2)^2 + (3-2)^2}{4}} \quad (4)$$

There will be two separate competitions. One of them will be used to measure the prediction performance and the other will be used to measure the time performance. You should submit the same code to both competitions, otherwise your submissions will not be valid and you will get no points. More details about this submission will be given by your TA.

4 Model Building and Experiments

The above description of UBCF and IBCF is a simple and general one. You are free to use different variations of these methods, especially you can try different similarity metrics (other than cosine). You can find more information on NBCF on many web resources. You are free to use any method you like (other than UBCF and IBCF) but you should implement at least UBCF or IBCF and show that it is working.

Since the ratings file that will be provided will be a large file, try to find the best (efficient) data structures and algorithms for applying UBCF and IBCF.

Before submitting your predictions, you might want to do offline experiments. In its simplest form you can split your data into training and test sets. The test set might be constructed by randomly selecting a subset of the rows (1000 rows for example) of the dataset provided and putting all the rest of the ratings to the training set. Then you can try to predict the ratings in the test set using the training set and calculate the MAE without making a submission. Note that you can make at most 3 submissions to HackerRank web site every day.

5 Requirements and What to Submit

- You can form teams of at least 3 at most 4 students. Team members can be from different sections.
- Codes should be written in C++ or Java. You are allowed to use C++ or Java standard library. No other library can be used.
- In the project report you should clearly explain the data structures and algorithms you used. Also, you should clearly write which parts of the project you completed. Designing efficient data structures and algorithms will lead to higher grades, so describe them in detail.
- You should submit a video recording in which every team member should explain his/her part. Recordings should be about 10-15 minutes long.

You should submit (to itslearning) the following as three separate files in the given formats. **Submissions in other formats will not be evaluated.** Every team should make a single submission so write team member names clearly at the top of your reports. There will be a penalty of -10 points for every late day after the due date.

- Allsource files. (zip format)
- Project report (PDF format)
- Video recording (mp4 format)

6 Grading

- **20 points** Project Report.
- **10 points** Video Recording.
- **40 points** Correctness. Lower $RMSE$ scores will get more points. If your $RMSE < 1.0$ you will immediately get 25 points and the rest of the points will be given according to the ranking formula given below with respect to $RMSE$. If your $RMSE \geq 1.0$ then you will only get points from project report and video recording.
- **30 points** Running time. If you successfully make a submission to HackerRank (within the time limit and with $RMSE < 1.0$) you will immediately get 15 points and the rest of the points will be given according to the ranking formula given below with respect to running time.

Ranking Formula

$$\text{Points}(i) = P_{\max} \cdot \frac{N - i}{N - 1}$$

where

- $P_{\max}$: 15 points (awarded to the first rank).
- N : Total number of groups (total ranks).
- i : Rank of the group (1 for the top rank, N for the last rank).

7 Cheating Policy

You are not supposed to use each other's source code. Also do not use source blocks of code from Internet, another person or from a textbook.

All the source codes will be filtered through a similarity analysis tool, which is known to be effective against many types of code copying and changing tricks. If a cheating is detected, these projects will be graded as 0 and also university regulations will apply.